

UNIVERZITET U BEOGRADU  
ELEKTROTEHNIČKI FAKULTET



**IMPLEMENTACIJA BASH SKRIPTI ZA UPRAVLJANJE  
PODSETNICIMA**  
Diplomski rad

Mentor:  
Prof. dr Zoran Čiča

Kandidat:  
Dunja Mihajlović 2018/0158

Beograd, jul 2023

# SADRŽAJ

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>SADRŽAJ .....</b>                                                            | <b>1</b>  |
| <b>1. UVOD.....</b>                                                             | <b>2</b>  |
| <b>2. LINUX.....</b>                                                            | <b>3</b>  |
| 2.1. BASH .....                                                                 | 3         |
| 2.2. STRUKTURA BASH SKRIPTE .....                                               | 4         |
| 2.3. PROSLEDIVANJE ARGUMENATA BASH SKRIPTI .....                                | 5         |
| 2.4. DOZVOLE .....                                                              | 5         |
| 2.5. IZVRŠAVANJE SKRIPTE.....                                                   | 6         |
| 2.6. PREGLED OSNOVNIH ALATA I TEHNIKA KORIŠĆENIH U SKRIPTAMA .....              | 6         |
| <b>3. SSH.....</b>                                                              | <b>10</b> |
| 3.1. KAKO RADI SSH? .....                                                       | 10        |
| 3.2. USPOSTAVLJANJE SSH VEZE IZMEĐU 2 VIRTUELNE MAŠINE.....                     | 13        |
| <b>4. SKRIPTA ZA UNOS I BRISANJE PODSETNIKA .....</b>                           | <b>16</b> |
| 4.1. PROGRAMSKI KOD SKRIPTE .....                                               | 16        |
| 4.2. KORISNIČKO UPUTSTVO I ANALIZA KODA .....                                   | 17        |
| <b>5. SKRIPTA ZA SLANJE NOTIFIKACIJA NA LOKALNI I/ILI UDALJENI RAČUNAR.....</b> | <b>21</b> |
| 5.1. PROGRAMSKI KOD SKRIPTE .....                                               | 21        |
| 5.2. KORISNIČKO UPUTSTVO I ANALIZA KODA .....                                   | 22        |
| <b>6. ZAKLJUČAK.....</b>                                                        | <b>25</b> |
| <b>LITERATURA.....</b>                                                          | <b>26</b> |
| <b>SPISAK SLIKA .....</b>                                                       | <b>27</b> |
| <b>SPISAK SKRAĆENICA .....</b>                                                  | <b>28</b> |

# 1. UVOD

Brzina, efikasnost i automatizacija su ključni zahtevi u gotovo svim sferama života danas. Kompjuterski sistemi i softverski alati postali su neizostavni za obavljanje različitih zadataka, bilo da je reč o administraciji sistema, obradi podataka, upravljanju resursima ili svakodnevnim rutinskim aktivnostima.

U tom kontekstu, Bash (*Bourne Again Shell*) scripting se ističe kao moćan alat za automatizaciju i olakšavanje upravljanja Linux i Unix operativnim sistemima. Bash je tekstualni *shell* koji pruža interaktivni interfejs za komunikaciju između korisnika i operativnog sistema. Njegov pravi potencijal dolazi do izražaja kada se koristi za pisanje skripti koje omogućavaju automatizaciju repetitivnih zadataka i izvršavanje niza komandi u logički povezanim sekvencama.

Cilj ovog rada je da se pruži detaljan uvid u proces kreiranja, konfigurisanja i korišćenja skripti za upravljanje podsetnicima, kao i uspostavljanje komunikacije između dva udaljena računara. U tu svrhu implementirane su dve Bash skripte: jedna za dodavanje i brisanje zapisa podsetnika, dok je druga zadužena za slanje notifikacija u vreme upisanih podsetnika.

Rad započinje pregledom korišćenih alata i tehnologija, pri čemu se pažnja posvećuje Linux operativnom sistemu, Bash interpretatoru i osnovama pisanja Bash skripti. Dat je i kratak pregled komandi koje su korišćene u izradi ovog rada. U narednom poglavlju obrađen je SSH protokol koji omogućava sigurnu komunikaciju između dva udaljena računara. Detaljno je opisan proces razmene ključeva, kao i postupak uspostavljanja veze između dva udaljena računara pomoću SSH protokola radi uspostavljanje sigurne komunikacije.

Glavni deo rada fokusira se na prikaz i objašnjenje kodova skripti. U četvrtom poglavlju predstavljena je skripta koja omogućava unos i brisanje podsetnika u odgovarajući .txt fajl. Posebna pažnja posvećena je formatiranju podataka u .txt fajlu kako bi se omogućila preglednost i laka manipulacija podsetnicima.

Peto poglavlje fokusira se na skriptu koja neprestano radi, prati sistemsko vreme i upoređuje ga sa zapisima u pomenutom .txt fajlu. Kada se vreme sistema poklopi sa nekim od zapisanih vremena unetih u podsetnik, skripta generiše notifikaciju koja se ispisuje na ekranu lokalnog i/ili udaljenog računara.

## 2. LINUX

Linux je besplatan operativni sistem otvorenog koda koji se temelji na UNIX-u. Otvoren (javno dostupan) kod je jedna od najvećih prednosti Linux-a i jedan od glavnih razloga njegove rasprostranjenosti. To znači da svi imaju slobodu da proučavaju, identifikuju greške, izvrše poboljšanja i unapređenja i da dele te promene sa ostalima. Zajednička saradnja i transparentnost u razvoju doprinose kontinuiranom poboljšanju, sigurnosti i pouzdanosti.

Brz odziv na bezbednosne propuste i njihovo redovno otkrivanje pruža visok nivo bezbednosti po kojoj je Linux poznat. Takođe, Linux podržava mnoge sigurnosne mehanizme i alate, kao što su *firewall*-i, enkripcija podataka i autentifikacija korisnika. Ovo pomaže u zaštiti sistema od napada i neovlašćenog pristupa.

Linux kernel (jezgro Linux operativnog sistema) pruža mogućnost podrške za različite drajvere. Drajveri su softverski moduli koji omogućavaju komunikaciju između kernela i specifičnih hardverskih komponenti računara. Ova fleksibilnost omogućava Linuxu da podržava širok spektar hardvera, od grafičkih kartica i mrežnih adaptera do skenera i štampača. Zbog svih navedenih karakteristika prisutan je gotovo svuda - na desktop računarima, serverima, u *cloud*-u, mobilnim, *Internet of Things* (IoT) uređajima, među drugim platformama. Linux je zapravo osnova za druge operativne sisteme, poput Androida. Stečeno iskustvo u radu s Linux-om se može primeniti na sve platforme koje podržava, a takvih je mnogo.

Linux podržava različite distribucije (operativne sisteme zasnovane na Linux jezgri) koje mogu imati različite upravljačke programe, sistemsku konfiguraciju i korisničko okruženje. Podržava virtuelizaciju omogućavajući pokretanje više virtuelnih mašina na istom fizičkom računaru. Za potrebe izrade ovog rada korišćene su dve Linux distribucije (virtuelne mašine) – *Kali Linux* i *Ubuntu 22.04*.

### 2.1. Bash

Shell je *programska ljuska* (engl. *shell*) koja predstavlja korisnički interfejs (tekstualni – CLI, ili grafički - GUI) za interakciju sa operativnim sistemom. To je sloj softvera koji omogućava korisniku da komunicira sa jezgrom operativnog sistema, pokreće komande i daje instrukcije operativnom sistemu da izvršava različite zadatke. Linux podržava različite shell interpretere, uključujući Bash.

Bash (*Bourne Again Shell*) je popularan tekstualni shell i skriptni jezik koji se koristi na Linux operativnom sistemu, kao i na drugim sistemima baziranim na Unix-u. Smešten je u *binaries* (*bin*) direktorijumu. Bash je podrazumevani shell na većini Linux distribucija i pruža niz komandi i funkcionalnosti koje omogućavaju korisnicima da izvršavaju različite operacije na sistemskim resursima.

Kada korisnik unese komandu u terminal, Bash shell interpretira tu komandu i prepoznaje je kao naredbu koju treba izvršiti. Shell zatim traži odgovarajući program ili funkcionalnost u operativnom sistemu koji će izvršiti tu komandu. Izvršavanje određene komande može rezultirati promenom stanja sistema, ispisivanjem rezultata u terminalu ili generisanjem grešaka.

Pored interaktivnog pristupa, Bash pruža mogućnost pisanja skripti, što znači da se sve komande koje se unose ručno u komandnu liniju mogu automatizovati kodom. Skripte se sastoje od niza komandi, logičkih struktura, petlji i drugih konstrukcija, čime se omogućava korisnicima da

obave više zadataka jednim pokretanjem skripte. Time se štedi vreme i smanjuje mogućnost grešaka. Korisnici na taj način mogu prilagoditi i personalizovati svoje okruženje, izvršiti složene operacije nad grupom fajlova, obraditi tekstualne podatke, upravljati sistemskim resursima i još mnogo toga. Bash skripte se mogu izvršavati iz komandne linije, ali i integrisati u sistemski proces pokretanjem ili raspoređivanjem na različite načine.

Na primer, za prilagođavanje i personalizaciju Bash sesije koriste se *.bash\_profile* i *.bashrc* konfiguracioni skript fajlovi. Pri prvom pokretanju shell-a za određenog korisnika izvršava se *.bash\_profile* skripta u kojoj se definišu trajne promenljive okruženja, pokretanje dodatnih skripti i druge komande koje se izvršavaju samo jednom pri pokretanju sesije. Pri svakoj novoj interaktivnoj sesiji, odnosno, pri svakom pokretanju terminala, učitava se *.bash\_rc* skripta u kojoj se definišu postavke okruženja, funkcije, alijasi i druga podešavanja koja su specifična za korisnika.

## 2.2. Struktura Bash skripte

Skripte su tekstualni fajlovi koji se kreiraju i uređuju pomoću različitih tekst editora (nano, vim, mousepad i drugi). Uobičajeno je da se skripte koje koriste Bash jezik označe ekstenzijom *.sh* radi prepoznavanja i jasnog identifikovanja da je u pitanju Bash skripta.

Bitno je da se skripta pravilno konfiguriše i izvrši putem Bash interpretera. To se postiže dodavanjem tzv. "*shebang*" linije (*#!/*) na početak skripte:



```
1 #!/bin/bash
?
```

Slika 1. shebang linija

Ova linija informiše operativni sistem da skripta treba da se izvrši korišćenjem Bash interpretera. Kada se *shebang* linija dodaje na početak skripte, operativni sistem automatski koristi Bash interpreter za izvršavanje te skripte, bez obzira na ekstenziju fajla. *Shebang* linija omogućava da se skripta izvrši direktno iz komandne linije, bez potrebe za eksplicitnim navođenjem interpretera pri svakom pokretanju. Takođe, omogućava da se skripta izvrši kao samostalna jedinica, nezavisno od okruženja ili trenutnog direktorijuma. Nakon toga sledi kod koji treba da omogući odgovarajuće funkcionalnosti skripte.

Bash omogućava definisanje promenljivih za čuvanje vrednosti. Promenljive se koriste za čuvanje podataka i mogu se manipulirati i koristiti u skriptama. Podržane su i kontrolne strukture za izvršavanje određenih blokova koda na osnovu uslova ili iteraciju kroz skup podataka poput petlji (*for*, *while*), uslova (*if*, *case*) i skakanja (*break*, *continue*, *exit*). Takođe, Bash pruža podršku za ulazne i izlazne tokove, što omogućava interakciju skripti sa korisnikom, kao i čitanje i pisanje podataka iz datoteka, kao i debugovanje skripti uz prikazivanje poruka o greškama i praćenje izvršavanja koda. Moguće je konfigurirati različite postavke, definisati korisničke funkcije u okviru skripti, što doprinosi fleksibilnosti i pruža mogućnost višestruke primene koda. Takođe, korisnici mogu prilagoditi izgled i ponašanje Bash prompta,

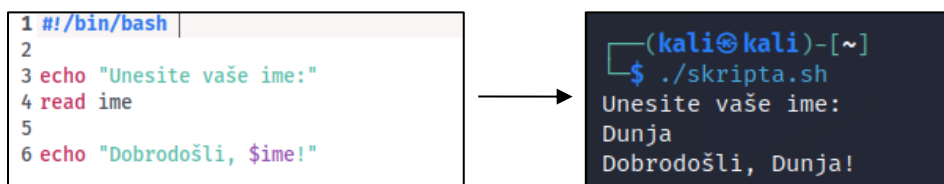
## 2.3. Prosleđivanje argumenata Bash skripti

- 1) Pozicioni argumenti u skriptama se odnose na argumente koje korisnik može proslediti prilikom pokretanja skripte. Oni se koriste za prosleđivanje ulaznih podataka skripti i omogućavaju da skripta bude fleksibilna i može se prilagoditi različitim scenarijima. Referišu se preko specijalnih varijabli: \$1, \$2, \$3, itd. Broj predstavlja poziciju argumenta, pri čemu \$1 predstavlja prvi argument, \$2 predstavlja drugi argument, i tako dalje. Mogu se koristiti za manipulaciju fajlovima, prosleđivanje opcija ili parametara skripti i prilagođavanje njihovog ponašanja na osnovu ulaznih vrednosti koje korisnik pruža pri pokretanju skripte. Slika 2 ilustruje primer pozicionih argumenata.



Slika 2. Pozicioni argumenti

- 2) Ulaz sa tastature - interaktivni unos putem komandi za unos podataka poput "read". Na primer, kao što je prikazano na slici 3 gde korisnik interaktivno unosi svoje ime.



Slika 3. Interaktivni unos

## 2.4. Dozvole

U Linux sistemu, dozvole se dodeljuju fajlovima kako bi se kontrolisalo ko ima pristup i ko može da ih izvrši, uređuje i čita. Postoje tri osnovne dozvole koje se mogu dodeliti fajlovima:

- 1) Izvršna dozvola (**x**): Ova dozvola omogućava izvršavanje fajla kao program. Ako fajl ima izvršnu dozvolu, možete ga pokrenuti kao skriptu ili izvršni program.
- 2) Dozvola za čitanje (**r**): Ova dozvola omogućava čitanje sadržaja fajla. Ako fajl ima dozvolu za čitanje, možete pristupiti njegovom sadržaju i čitati ga.
- 3) Dozvola za pisanje (**w**): Ova dozvola omogućava pisanje ili menjanje sadržaja fajla. Ako fajl ima dozvolu za pisanje, možete ga menjati ili ažurirati.

Komanda **ls -l putanja\_do\_direktorijuma** prikazuje dodeljene dozvole za sve fajlove u određenom direktorijumu.

Dozvole za izvršavanje skripti u Bash-u odnose se na mogućnost pokretanja skripte kao izvršnog programa. Stoga, neophodno je dodeliti izvršnu dozvolu skripti, koristite komandu **chmod** u kombinaciji sa **+x** (+ označava dodeljivanje, a x izvršnu dozvolu) i putanjom do skripte.

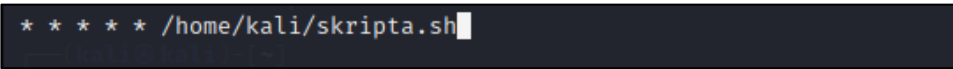


Slika 4. Dodeljivanje izvršne dozvole

## 2.5. Izvršavanje skripte

Skripta se kao izvršni program pokreće navođenjem **kompletne putanje** do nje (`/putanja_do_skripte.sh`), ili uvođenjem odgovarajućeg alijasa.

Skripta se može izvršiti jednom, po potrebi, upisivanjem u prompt ili se može omogućiti njeno periodično izvršavanje u određenim vremenskim intervalima. Za ovu namenu, često se koristi **cron**. Cron je vremenski upravljač (*engl. time-based job scheduler*) koji se koristi za automatizaciju periodičnih zadataka. On omogućava pokretanje skripti, programa ili komandi u određenim vremenskim intervalima, Funkcioniše tako što korisnici definišu cron zadatke, koji se zatim unose u cron tablicu. Cron tablica sadrži raspored izvršavanja za svakog korisnika i određuje kada će se određeni zadaci izvršiti. Otvara se pomoću komande **crontab -e**, nakon čega se na kraj fajla dodaje linija.



```
* * * * * /home/kali/skripta.sh
```

Slika 5. Zapis cron zadatka

Cron zadaci se definišu pomoću cron sintakse koja se sastoji od pet polja (\*): minut, sat, dan u mesecu, mesec i dan u nedelji, respektivno. Svako polje može da sadrži specifične vrednosti ili specijalne znakove koji definišu raspored izvršavanja.

U konkretnom primeru, *skripta.sh* će se izvršavati svakog minuta, jer \* u polju označava da se koriste sve moguće vrednosti za konkretno polje.

Druga mogućnost je upotreba beskonačne *while* petlje u kombinaciji sa *sleep* komandom:



```
1 #!/bin/bash
2
3 while true; do
4     #kod koji se periodicno izvorsava
5     sleep 60
6 done
```

Slika 6. Periodično izvršavanje skripte pomoću beskonačne petlje

*Sleep 60* označava da se kod izvršava svakih 60 sekundi, ovaj interval se može menjati u zavisnosti od potreba. Prednost ovog pristupa je u tome što je skriptu lakše kontrolisati i prekinuti, međutim, komandna linija nije slobodna za dalji unos komandi. Kako bi se omogućilo da se skripta izvršava u pozadini neophodno je dodati na kraj (u liniji 6 nakon *done*) znak **&** čime će komandna linija biti slobodna za korišćenje. Pomoću komande *kill PID* (process ID) moguće je prekinuti skriptu koja se izvršava u pozadini.

Kada su u pitanju zadaci koji se periodično izvršavaju u kratkim vremenskim intervalima, važno je voditi računa o opterećenju sistema i adekvatnoj upotrebi resursa kako ne bi došlo do degradacije performansi.

## 2.6. Pregled osnovnih alata i tehnika korišćenih u skriptama

U nastavku je dat kratak pregled komandi, kontrolnih struktura i drugih tehnika i alata korišćenih u izradi skripti za unos, brisanje i slanje podsetnika.

### ❖ Komande:

- **echo** - prikaz teksta na standardnom izlazu

- **read** - ugrađena komanda koja se koristi za čitanje korisničkog unosa sa standardnog ulaza. Ona čita liniju teksta koju korisnik unese i dodeljuje je promenljivoj. Opcije:  
**-r** - koristi za očuvanje tzv. *raw input*-a, kada uneti tekst treba da bude tretiran doslovno, bez interpretacije specijalnih znakova  
**-p "prompt"** – koristi se za postavljanje prilagođenog korisničkog upita (prompt) koji će biti prikazan pre unosa korisnika.
- **cat datoteka** – prikaz sadržaja datoteke
- **sed** - za manipulaciju tekstualnim podacima. Koristi se za transformaciju teksta, zamenu reči, linija ili znakova, brisanje ili dodavanje teksta, i mnoge druge operacije. Osnovna struktura sed komande je sledeća:

*sed [opcije] 'akcija' [datoteka]*

- **awk** – moćan, fleksibilan alat za obradu teksta sa relativno jednostavnom sintaksom. Koristi se za filtriranje redova, agregiranje, sumiranje, filtriranje i preoblikovanje podataka, manipulaciju stringovima i dr. Osnovna struktura komande je sledeća:

*awk [opcije] 'izbor {program}' [ulazne\_datoteke],*

gde su opcije:

- F 'separator'** – definiše se separator kolona u fajlu, što omogućava izdvajanje određenih polja, zamenu mesta, brisanje itd.
- v rec="\$ime"** – definiše vrednost varijable *rec* koja se može koristiti u okviru awk komande

- **mv** – za premeštanje i preimenovanje direktorijuma i datoteka
- **cut datoteka** - izdvajanje i manipulacija delova teksta iz linija ili polja teksta.  
**-d** definiše se delimiter teksta “|”  
**-f** označava polje teksta koje je potrebno izdvojiti. Na primer -f2 označava da se izdvaja drugo polje teksta
- **date [opcija]** - prikaz trenutnog vremena i datuma. Pruža mogućnosti formatiranja vremena i datuma prema specifičnim zahtevima kroz korišćenje kontrolnih karaktera. Na primer, "%Y" predstavlja četvorocifrenu godinu, "%m" predstavlja (dvocifreni) broj meseca, "%d" predstavlja (dvocifreni) broj dana, itd.
- **notify-send** komanda se koristi za slanje GUI notifikacija na korisničkom ekranu. Ova komanda omogućava slanje jednostavnih, kratkih obaveštenja ili poruka korisniku.  
**-i** - dodavanje ikonice GUI notifikacije  
**-c** specificiranje kategorije notifikacije  
**-u** – specificiranje nivoa prioriteta (*urgency level*) notifikacije

Za sve dostupne opcije, može se pogledati dokumentacija Bash komandi koristeći **man** ili **help** komandu.

- ❖ **Pajpovi (pipes)** u Bash-u su mehanizam koji omogućava preusmeravanje izlaza jedne komande kao ulaz za drugu komandu, bez potrebe za upisivanjem podataka u privremene



datoteke. Označavaju se znakom "|" (vertikalna crta) i koriste se za povezivanje komandi u jedan niz komandi koje se izvršavaju jedna za drugom. Na primer, u liniji:

```
cat datoteka.txt | awk '{print $2}'
```

prikaz sadržaja datoteke koristi se kao ulaz za awk komandu koja ispisuje drugu kolonu datoteke.

- ❖ **Redirekcija ulaza i izlaza** omogućava preusmeravanje toka podataka između komandi i datoteka.
  - „<“ - preusmeravanje ulaza komande iz datoteke, umesto ručnog unosa
  - „>“ - preusmeravanje izlaza komande u datoteku, umesto ispisa na ekranu, pri čemu se prvobitni sadržaj datoteke briše i u potpunosti zamenjuje izlazom komande
  - „>>“ – izlaz komande se dodaje na kraj datoteke
  - „2>“ – preusmeravanje standardnih grešaka (*stderr*) u datoteku
- ❖ **Kontrolne strukture** omogućavaju upravljanje tokom izvršavanja skripte na osnovu uslova ili izbora. Sintaksa je data na slici 7.

```
3 # Case
4 case $varijabla in
5     value1)
6         # Kod koji se izvršava ako je varijabla=value1
7         ;;
8     value2)
9         # Kod koji se izvršava ako je varijabla=value2
10        ;;
11    *)
12        # Kod koji se izvršava ako nijedna vrednost ne odgovara
13        ;;
14 esac
15
16 # If-elif-else
17 if [ uslov1 ]; then
18     # Kod koji se izvršava ako je uslov1 tačan
19 elif [ uslov2 ]; then
20     # Kod koji se izvršava ako je uslov2 tačan
21 else
22     # Kod koji se izvršava ako ni jedan uslov nije tačan
23 fi
24
25 # While petlja
26 while [ uslov ]; do
27     # Kod koji se izvršava dok je uslov tačan
28 done
```

Slika 7. Sintaksa case, if, while

- ❖ **Variable** su simbolički identifikatori koji se koriste za čuvanje i pristupanje vrednostima poput brojeva, teksta, logičkih vrednosti ili bilo kojih drugih podataka. Bash ne zahteva eksplicitno navođenje tipa podatka prilikom uvođenja nove varijable. Deklarišu se dodeljivanjem vrednosti imenu varijable (*broj=5*). Vrednosti varijable se pristupa dodavanjem prefiksa „\$“ ispred njenog imena. U zavisnosti od doseg ili vidljivosti u okviru programa ili skripte, razlikuju se lokalne (definisane i dostupne samo u okviru određenih okruženja ili blokova koda) i globalne varijable (definisane i dostupne za sve procese i sesije u sistemu).

- **IFS (Internal Field Separator)** je specijalna varijabla u Bash-u koja se koristi za definisanje separatora koji se koriste prilikom razdvajanja stringova na polja. Podrazumevana vrednost IFS-a je razmak, tab, novi red, ali ona se može proizvoljno definisati jednostavnom dodelom nove vrednosti promenljivoj. Kada se IFS koristi u skriptama zajedno sa naredbom *read* ili prilikom razdvajanja teksta unutar skripte, vrednost IFS se koristi za određivanje mesta razdvajanja. Kada se string podeli na polja koristeći IFS separator, svako polje se tretira kao zaseban element.
- ❖ **getopts** je ugrađena Bash funkcija koja se koristi za definisanje opcija i argumenata u skripti. Opcije skripte omogućavaju da se prilagodi ponašanje skripte prema potrebama korisnika i specificiraju dodatne informacije ili postavke koje skripta koristi prilikom izvršavanja.

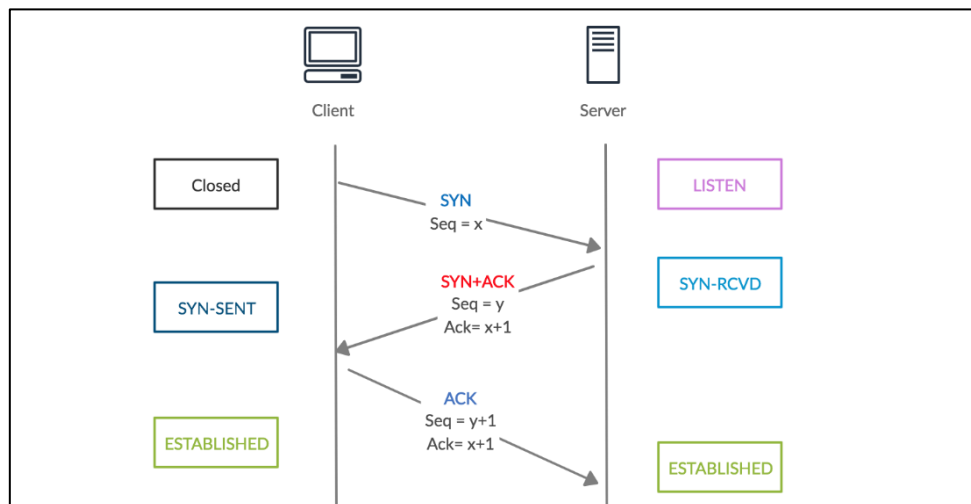
## 3. SSH

SSH (*Secure Shell*) je kriptografski mrežni protokol koji omogućava sigurno udaljeno upravljanje i prenos podataka između dva računara putem (nesigurne) mreže. Za razliku od Telnet-a, SSH koristi tehnike kriptografije kojima omogućava da se enkriptuju svi podaci koji se razmenjuju između dva računara putem mreže. Time se obezbeđuje siguran prenos i sprečava neovlašćeni pristup podacima. Neovlašćeni korisnici i dalje mogu videti pakete koji se šalju putem mreže, pokušavaju da ih presreću i kopiraju, ali nemaju pristup sadržaju paketa.

SSH se zasniva na klijent-server modelu. Klijent je računar sa kojeg se uspostavlja SSH veza i sa kojeg se šalju zahtevi serveru. Server je računar koji prihvata SSH veze i izvršava komande ili omogućava prenos podataka za klijenta. SSH se obično koristi za prijavljivanje na udaljenu mašinu i izvršavanje komandi, ali takođe podržava tuneliranje, prosleđivanje TCP portova i X11 konekcija; može prenositi datoteke koristeći povezane SSH protokole za prenos datoteka (SFTP) ili sigurnu kopiju (SCP).

### 3.1. Kako radi SSH?

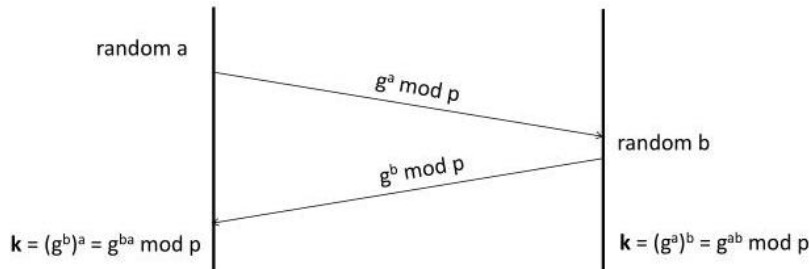
SSH podrazumevano funkcioniše na TCP portu 22 (može se promeniti ako je potrebno).



Slika 8. Three-way handshake

- 1) Uspostavlja se TCP konekcija trostrukim rukovanjem prikazanim na slici 8. Klijent šalje SYN paket serveru kako bi započeo komunikaciju. SYN (*Synchronize*) paket sadrži inicijalne podatke, uključujući broj sekvence koji identifikuje početak podataka. Server prima SYN paket od klijenta i šalje nazad SYN-ACK (*Synchronize-Acknowledge*) paket koji potvrđuje da je SYN paket primljen. Ovaj paket takođe sadrži broj sekvence za server i broj potvrde koji označava potvrdu broja sekvence klijenta. Klijent prima SYN-ACK paket od servera i šalje ACK (*Acknowledge*) paket kao potvrdu nakon čega se veza uspostavlja i oba uređaja mogu početi sa razmenom podataka.
- 2) Razmena verzija protokola radi usklađivanja tokom sesije. Ako klijentu odgovara jedna od verzija protokola koje podržava server, nastavlja se uspostava veze.

- 3) Dogovor o algoritmima za enkripciju, kompresiju podataka i autentifikaciju. Obe strane razmenjuju listu podržanih algoritama i dogovaraju se o zajedničkim algoritmima i parametrima koji će biti korišćeni.
- 4) Razmena ključeva – obe strane koriste verziju Diffie-Hellman algoritma za pregovaranje o ključu sesije (*session-key*) koji se za šifrovanje podataka koristi tokom cele sesije. Diffie-Hellman algoritam omogućava svakoj strani da kombinuje svoje privatne podatke sa javnim podacima druge strane kako bi došli do istog tajnog ključa na sledeći način:
  - Obe strane su se dogovorile o dva javno poznata broja – prost broj  $p$  i generator  $q$ .
  - Svaka strana generiše svoj privatni ključ, tj. tajni broj koji se ne deli sa drugom stranom ( $a$  i  $b$  na slici 9)
  - Izračunavanje javnih ključeva: Svaka strana izračunava svoj javni ključ tako što generator  $g$  podigne na stepen tajnog broja/ključa, a zatim se rezultat deli prostim brojem  $p$ . Ostatak pri deljenju predstavlja javni ključ.
  - Razmena javnih ključeva
  - Izračunavanje zajedničkog tajnog ključa: obe strane koriste primljeni javni ključ i svoj privatni ključ kako bi izračunale tajni ključ. Obe strane dobijaju isti zajednički ključ kojim se vrši simetrično enkriptovanje i dekriptovanje podataka tokom sesije (*session-key*). Na ovaj način se obezbeđuje da treća strana ne može nikako doći do tajnog ključa kojim se enkriptuju svi podaci u SSH sesiji, jer bez obzira na javnu razmenu ključeva, privatni ključevi su tajni i ne razmenjuju se između strana, tako da je krajnji simetrični ključ gotovo nemoguće razbiti.



Slika 9. Diffie-Hellman algoritam

- 5) Klijent šalje serveru kriptovani ključ sesije.
- 6) Server dekriptuje ključ sesije i potvrđuje uspostavu sigurne veze.
- 7) Autentifikacija - klijent i server mogu razmenjivati autentifikacione podatke kako bi proverili identitet jedni drugima.

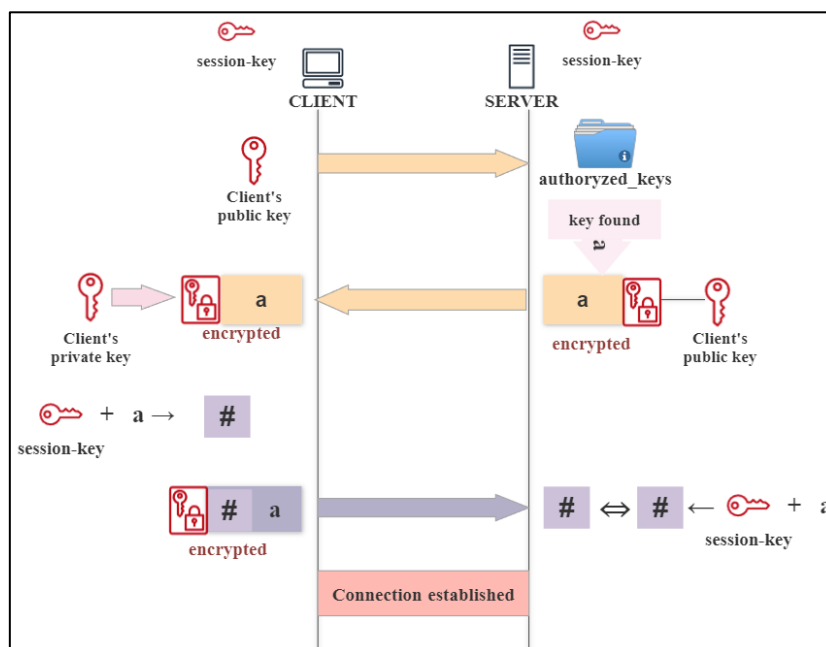
Postoje različite metode autentifikacije kako bi se osiguralo da samo ovlašćeni korisnici mogu pristupiti računaru putem SSH veze. Autentifikacija na osnovu lozinke je podrazumevana, pri čemu server traži od klijenta lozinku za nalog kojim pokušava da se prijavi. Iako se lozinka šalje enkriptovana, ovaj metod autentifikacije se ne preporučuje jer se lozinke normalnih dužina obično mogu lako probiti pomoću automatizovanih skript programa.

Asimetrični ključevi su najpopularniji metod autentifikacije. Korisnik koristi par asimetričnih ključeva, što znači da privatni i javni ključ imaju različite funkcije. Javni ključ

se slobodno može deliti i on se koristi za šifrovanje podataka, dok se privatni ključ ne sme deliti i jedino njime se mogu dešifrovati podaci na prijemu.

- Autentifikacija servera: Server šalje svoj javni ključ klijentu. Ukoliko se u klijentovoj datoteci *known\_hosts* ne nalazi zapis o javnom ključu servera, klijent će obično prikazati korisniku poruku upozorenja da nije moguće izvršiti autentifikaciju servera i upit da li želi da nastavi sa konekcijom. Ukoliko „veruje“ serveru, korisnik će potvrdno odgovoriti na poruku i zapis javnog ključa servera će se automatski dodati u datoteku. Ova poruka predstavlja sigurnosnu meru koja obezbeđuje da se klijent povezuje samo sa pouzdanim serverima.
- Autentifikacija klijenta: Klijent šalje javni ključ serveru. Server proverava datoteku *authorized\_keys* u kojoj se nalaze javni ključevi klijenata koji su ovlašćeni da se prijave. Ukoliko se u datoteci nalazi javni ključ klijenta, server generiše slučajni broj i koristi taj javni ključ da enkriptuje broj. Ako klijent zaista ima odgovarajući privatni ključ za dešifrovanje, moći će da otkrije originalni broj. Klijent kombinuje dešifrovani broj sa simetričnim ključem sesije kako bi se generisala heš (*hash*) vrednost. Heš se potom šalje zajedno sa enkriptovanom porukom. Server koristi isti deljeni ključ sesije i originalni broj koji je poslao klijentu da bi i sam izračunao heš. Ako se primljena i poslata vrednost poklapaju, dokazano je da klijent zaista poseduje odgovarajući privatni ključ i klijent je autentifikovan.

U SSH, heš vrednosti se uglavnom koriste za proveru integriteta podataka (da podaci nisu slučajno ili namerno izmenjeni) i za proveru autentičnosti komunikacije. Za heširanje podataka u SSH protokolu se često koriste algoritmi kao što su MD5, SHA-1, SHA-2 i dr. Algoritam se bira tako što se prolazi kroz listu prihvatljivih izbora MAC-a klijenta. Prvi podržani algoritam sa te liste koji server podržava će se koristiti.



Slika 10. Autentifikacija klijenta

- 8) Sesija – nakon uspešne autentifikacije uspostavljena je SSH sesija. Klijent i server komuniciraju preko sigurnog kanala, a podaci se šalju koristeći simetrično šifrovanje sa ključem sesije.

Na slici 11, dat je prikaz snimanja paketa tokom uspostave SSH veze između *Kali klijenta* (192.168.1.32) i *Ubuntu servera* (192.168.1.31).

| No. | Source       | Destination  | Protocol | Info                                                                  |
|-----|--------------|--------------|----------|-----------------------------------------------------------------------|
| 4   | 192.168.1.32 | 192.168.1.31 | TCP      | 48268 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=3   |
| 5   | 192.168.1.31 | 192.168.1.32 | TCP      | 22 → 48268 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PER   |
| 6   | 192.168.1.32 | 192.168.1.31 | TCP      | 48268 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3996124747 TSecr   |
| 7   | 192.168.1.32 | 192.168.1.31 | SSHv2    | Client: Protocol (SSH-2.0-OpenSSH_9.2p1 Debian-2)                     |
| 8   | 192.168.1.31 | 192.168.1.32 | TCP      | 22 → 48268 [ACK] Seq=1 Ack=33 Win=65152 Len=0 TSval=2381536170 TSecr  |
| 9   | 192.168.1.31 | 192.168.1.32 | SSHv2    | Server: Protocol (SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.7)            |
| 10  | 192.168.1.32 | 192.168.1.31 | TCP      | 48268 → 22 [ACK] Seq=33 Ack=42 Win=64256 Len=0 TSval=3996124778 TSecr |
| 11  | 192.168.1.32 | 192.168.1.31 | SSHv2    | Client: Key Exchange Init                                             |
| 12  | 192.168.1.31 | 192.168.1.32 | SSHv2    | Server: Key Exchange Init                                             |
| 13  | 192.168.1.32 | 192.168.1.31 | TCP      | 48268 → 22 [ACK] Seq=1537 Ack=1098 Win=64128 Len=0 TSval=3996124782   |
| 14  | 192.168.1.32 | 192.168.1.31 | SSHv2    | Client: Elliptic Curve Diffie-Hellman Key Exchange Init               |
| 15  | 192.168.1.31 | 192.168.1.32 | SSHv2    | Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys,   |
| 16  | 192.168.1.32 | 192.168.1.31 | TCP      | 48268 → 22 [ACK] Seq=1585 Ack=1534 Win=64128 Len=0 TSval=3996124799   |
| 17  | 192.168.1.32 | 192.168.1.31 | SSHv2    | Client: New Keys                                                      |
| 18  | 192.168.1.31 | 192.168.1.32 | TCP      | 22 → 48268 [ACK] Seq=1534 Ack=1601 Win=64128 Len=0 TSval=2381536275   |
| 19  | 192.168.1.32 | 192.168.1.31 | SSHv2    | Client: Encrypted packet (len=44)                                     |
| 20  | 192.168.1.31 | 192.168.1.32 | TCP      | 22 → 48268 [ACK] Seq=1534 Ack=1645 Win=64128 Len=0 TSval=2381536276   |
| 21  | 192.168.1.31 | 192.168.1.32 | SSHv2    | Server: Encrypted packet (len=44)                                     |
| 22  | 192.168.1.32 | 192.168.1.31 | SSHv2    | Client: Encrypted packet (len=68)                                     |
| 23  | 192.168.1.31 | 192.168.1.32 | SSHv2    | Server: Encrypted packet (len=52)                                     |
| 24  | 192.168.1.32 | 192.168.1.31 | SSHv2    | Client: Encrypted packet (len=500)                                    |
| 25  | 192.168.1.31 | 192.168.1.32 | SSHv2    | Server: Encrypted packet (len=460)                                    |
| 26  | 192.168.1.32 | 192.168.1.31 | SSHv2    | Client: Encrypted packet (len=908)                                    |
| 27  | 192.168.1.31 | 192.168.1.32 | SSHv2    | Server: Encrypted packet (len=28)                                     |
| 28  | 192.168.1.32 | 192.168.1.31 | SSHv2    | Client: Encrypted packet (len=112)                                    |
| 29  | 192.168.1.31 | 192.168.1.32 | TCP      | 22 → 48268 [ACK] Seq=2118 Ack=3233 Win=64128 Len=0 TSval=2381536352   |

Slika 11. Wireshark PCAP – uspostava SSH sesije (TCP port 22)

### 3.2. Uspostavljanje SSH veze između 2 virtuelne mašine

U nastavku je dat prikaz komandi za uspostavu veze između 2 virtuelne mašine pomoću SSH:

- 1) Prvi korak je instalacija SSH klijenta i servera na virtuelnim mašinama – slika 12.

```
(kali@kali)-[~]
$ sudo apt install openssh-server openssh-client
```

Slika 12. Instalacija SSH servera i klijenta

- 2) Provera statusa SSH servera: radi provere da li je SSH server pokrenut na udaljenom računaru koristiti komandu `systemctl status sshd` ili `service ssh status`. Ove komande će prikazati status SSH servera – slika 13.

```
ubuntu@ubuntu:~/Desktop$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: en
   Active: active (running) since Mon 2023-06-12 19:43:58 CEST; 3min 59s ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 3720 (sshd)
     Tasks: 1 (limit: 2284)
    Memory: 1.0M
   CGroup: /system.slice/ssh.service
           └─3720 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
```

Slika 13. Provera statusa SSH

- 3) Podešavanje konfiguracije SSH servera: Konfiguraciona datoteka SSH servera se obično nalazi na lokaciji `/etc/ssh/sshd_config`. U datoteci se pomoću tekst editor-a (npr. nano) mogu podesiti opcije kao što su port, dozvoljene metode autentifikacije, enkripcija i drugi

parametri. Izmene default postavki se vrše jednostavnim brisanjem # ispred opcije koju je potrebno promeniti i unosom odgovarajućih vrednosti.

- 4) Podešavanje *firewall*-a: Ukoliko se koristi *firewall* na udaljenom računaru, potrebno je otvoriti SSH port kako bi bio omogućen pristup SSH serveru. Korišćenjem ***sudo ufw allow ssh*** otvara se podrazumevani SSH port (22) – slika 14.

```
ubuntu@ubuntu:~/Desktop$ sudo ufw allow ssh
Rules updated
Rules updated (v6)
```

Slika 14. Otvaranje SSH porta 22

- 5) Generisanje asimetričnih ključeva: Radi sigurne autentifikacije putem SSH-a, možete generisati javni-privatni ključ par na lokalnom računaru. U konkretnom primeru sa slike 15 generišu se RSA ključevi. Simetrični ključevi sesije se generišu dinamički tokom uspostavljanja SSH sesije.

```
(kali@kali)-[~]
$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kali/.ssh/id_rsa): /home/kali/.ssh/id_rsa1
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_rsa1
Your public key has been saved in /home/kali/.ssh/id_rsa1.pub
The key fingerprint is:
SHA256:qoDnEmzqkkKoJLXQqXYztEnGfZnPeyh+gz4jXkWWMuE kali@kali
The key's randomart image is:
+--[RSA 3072]--+
|      .      |
|      . . .   |
|...   Eo+    |
|..o.  +=     |
|*ooo. o.S    |
|+%o   .+     |
|O+*   .o o   |
|O+ o.* = .   |
|=oo.=o= o    |
+--[SHA256]--+
```

Slika 15. Generisanje RSA ključeva

- 6) Prilikom prvog prijavljivanja na server potrebno je izvršiti kopiranje javnog ključa na SSH server kao što je prikazano na slici 16.

```
(kali@kali)-[~]
$ ssh-copy-id username@remote_host
```

Slika 16. Kopiranje ključa na SSH server

Username – korisničko ime udaljenog korisnika

Remote\_host – IP adresa udaljenog korisnika ili DNS ime

Komandom *ifconfig* saznajemo IP adresu na udaljenom računaru.

- 7) Povezivanje na udaljeni računar korišćenjem ssh komande ***ssh username@remote\_host*** će uspostaviti SSH sesiju sa udaljenim računarom "adresa servera" sa IP adresom ili domenom udaljenog računara. Ukoliko se koristi drugi SSH port, može se specificirati

opcijom `-p broj_porta`. (Na slici 11 dat je prikaz razmene paketa nakon izvršavanja ove komande)

- 8) Nakon uspešne SSH veze, mogu se izvršavati komande na udaljenom računaru putem Bash-a i obavljati razni zadaci poput upravljanja datotekama i izvršavanja skripti, kao što je prikazano na slici 17.

```
(kali㉿kali)-[~]
$ ssh ubuntu@192.168.1.31
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-75-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Expanded Security Maintenance for Applications is not enabled.

20 updates can be applied immediately.
11 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '22.04.2 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Your Hardware Enablement Stack (HWE) is supported until April 2025.
Last login: Sun Jul  2 23:27:54 2023 from 192.168.1.32
ubuntu@ubuntu:~$
```

Slika 17. Uspešna uspostava SSH veze



## 4. SKRIPTA ZA UNOS I BRISANJE PODSETNIKA

U ovom poglavlju biće izloženo korisničko uputstvo uz objašnjenje koda i prikaz rada skripte koja se koristi za unos i brisanje podsetnika iz tekstualnog fajla. Ova skripta donosi benefite u smislu preciznog formatiranja teksta, postavljanja separatora i formiranja kolona radi organizacije podataka, čime se postiže veća preglednost i lakša manipulacija podacima.

### 4.1. Programski kod skripte

```
1 #!/bin/bash
2
3 proveravreme() {
4
5 pattern="^([01]?[0-9]|2[0-3]):[0-5][0-9]$"
6
7 if [[ ! $1 =~ $pattern ]]; then
8     echo "Greska: Pogresan format vremena"
9     exit 1
10 fi
11 }
12
13 proveravreme() {
14 if [ -z "$1" ]; then
15     echo "Greska! Nije unet trazeni podatak"
16     exit 1
17 fi
18 }
19
20 function dodaj() {
21     read -rp "Naziv podsetnika " naziv
22     proveravreme $naziv
23     read -rp "Opis podsetnika " opis #opciono
24     read -p "Vreme (hh:mm) " vreme ;
25     proveravreme $vreme
26     read -p "Vreme notifikacije (hh:mm) " vreme_notif
27     proveravreme $vreme_notif
28
29     red="$vreme_notif|$vreme|$naziv|$opis"
30     echo "$red" >> podsetnik.txt
31     echo "Podsetnik uspesno dodat."
32 }
33
34 function obrisi(){
35     read -rp "Naziv podsetnika koji treba obrisati " naziv
36     proveravreme $naziv
37     awk -F "|" -v rec="$naziv" 'tolower($3) == tolower(rec) { next } { print }' podsetnik.txt > tmp.txt
38     mv tmp.txt podsetnik.txt
39     echo "Podsetnik uspesno obrisan"
40 }
41
42 echo "1 - dodaj podsetnik"
43 echo "2 - obrisi podsetnik"
44 echo "3 - pregled svih podsetnika"
45 while true ; do
46     read -p "Izaberite opciju (ili upisite 4 za izlaz): " opcija
47
```

```

48     case $opcija in
49         4) break ;;
50         1) dodaj ;;
51         2) obrisi ;;
52         3) cat podsetnik.txt ;;
53         *) echo "Neispravan unos."
54             continue ;;
55     esac
56 done

```

Slika 18. Programski kod skripte za unos, brisanje i pregled podsetnika - *podsetnik.sh*

## 4.2. Korisničko uputstvo i analiza koda

Pri pokretanju skripte, pojavljuje se meni iz kog je potrebno izabrati odgovarajuću opciju – slika 19. Korišćenjem beskonačne *while* petlje omogućeno je izvršavanje više aktivnosti (pregled, dodavanje i/ili brisanje podsetnika) pri samo jednom pokretanju skripte, tj. bez ponovnog pojavljivanja menija. U slučaju izbora nepostojeće opcije, korisniku se ponovo omogućava izbor opcije. Korisnik će nakon svake izvršene akcije biti obavešten o tome da li je zapis podsetnika uspešno unet ili obrisan.

```

39 echo "1 - dodaj podsetnik"
40 echo "2 - obrisi podsetnik"
41 echo "3 - pregled svih podsetnika"
42 while true ; do
43     read -p "Izaberite opciju (ili upisite 4 za izlaz): " opcija
44
45     case $opcija in
46         4) break ;;
47         1) dodaj ;;
48         2) obrisi ;;
49         3) cat podsetnik.txt ;;
50         *) echo "Neispravan unos."
51             continue ;;
52     esac
53 done

```

```

(kali@kali)-[~]
$ ./podsetnik.sh
1 - dodaj podsetnik
2 - obrisi podsetnik
3 - pregled svih podsetnika
Izaberite opciju (ili upisite 4 za izlaz): z
Neispravan unos.
Izaberite opciju (ili upisite 4 za izlaz):

```

Slika 19. Pokretanje skripte, prikaz menija i primer pogrešnog izbora opcije

U slučaju izbora opcije 1, izvršava se funkcija *dodaj*, prikazana na slici 20, pri čemu će se tražiti unos odgovarajućih podataka – naziv, opis, vreme i vreme prikaza notifikacije.

```

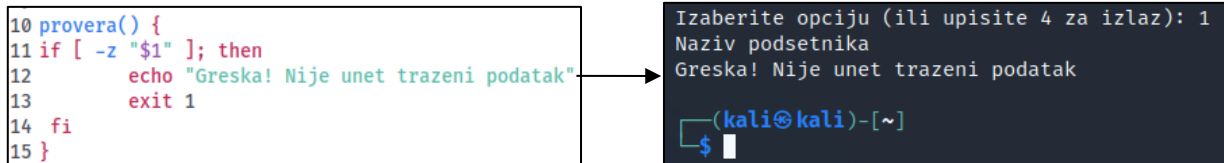
17 function dodaj() {
18     read -rp "Naziv podsetnika " naziv
19     prover_a $naziv
20     read -rp "Opis podsetnika " opis #opciono
21     read -p "Vreme (hh:mm) " vreme ;
22     prover_a_vreme $vreme
23     read -p "Vreme notifikacije (hh:mm) " vreme_notif
24     prover_a_vreme $vreme_notif
25
26     red="$vreme_notif|$vreme|$naziv|$opis" # konstruisanje reda za podsetnik
27     echo "$red" >> podsetnik.txt # ubacivanje reda u podsetnik.txt
28     echo "Podsetnik uspesno dodat."
29 }

```

Slika 20. Funkcija za dodavanje podsetnika

Pri unosu svakog (obaveznog) podatka vrši se odgovarajuća *provera* unetog sadržaja pozivom funkcije – slika 21. Opis podsetnika je opciono podatak, stoga nije potrebno izvršiti proveru unosa. Neispravan unos bilo kog podatka rezultira prekidom izvršavanja skripte. U slučaju unosa naziva

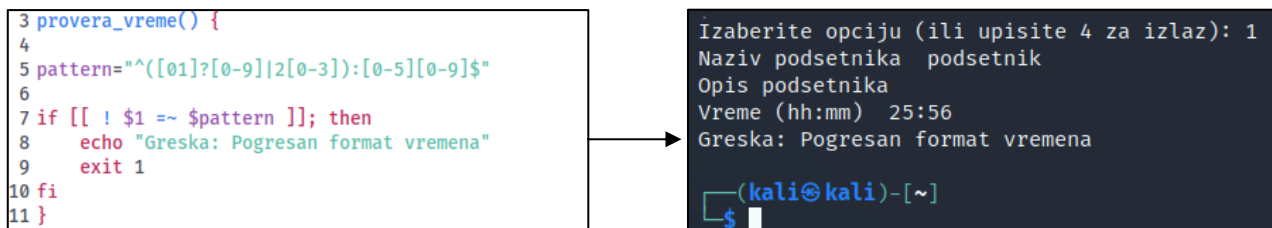
podsetnika, neispravan unos podrazumeva „prazan“ unos. Izvršava se funkcija *provera* kojoj se prosleđuje vrednost promenljive *naziv*.



Slika 21. Provera unosa naziva podsetnika

Koristi se *if-else* konstrukcija za proveru uslova. "-z" je uslovni operator koji proverava da li je vrednost prazna (nula dužine). *\$1* predstavlja prvi argument koji je prosleđen skripti.

Neispravan unos *vremena* i *vremena notifikacije* podrazumeva da vreme nije uneto u formatu hh:mm, pri čemu hh uzima vrednosti 00-23, dok mm uzima vrednosti 00-59. U okviru promenljive *pattern* definisan je šablon koji treba da ispuni zapis unetog vremena – slika 22.



Slika 22. Provera formata vremena

Formiraju se redovi u okviru fajla *podsetnik.txt* koji predstavljaju zapise podsetnika u formatu: *VREME\_NOTIFIKACIJE* | *VREME* | *NAZIV\_PODSETNIKA* | *OPIS\_PODSETNIKA*. Važno je napomenuti da korisnik pri unosu naziva i opisa podsetnika može uneti proizvoljan tekst, slova, brojeve i sve znakove osim „|“ uspravne linije koja se u fajlu koristi kao separator kolona.

U nastavku je dat prikaz kompletnog procesa pokretanja skripte i kreiranja zapisa podsetnika tekstualnom fajlu. Izborom opcije 3 prikazuje se kompletan sadržaj *podsetnik.txt* fajla. Ova opcija omogućava da korisnik prilikom korišćenja skripte može proveriti da li su uneti ispravni podaci.

```

L$ ./podsetnik.sh
1 - dodaj podsetnik
2 - obriši podsetnik
3 - pregled svih podsetnika
Izaberite opciju (ili upisite 4 za izlaz): 1
Naziv podsetnika HR intervju
Opis podsetnika online webex
Vreme (hh:mm) 14:00
Vreme notifikacije (hh:mm) 13:30
Podsetnik uspesno dodat.
Izaberite opciju (ili upisite 4 za izlaz): 1
Naziv podsetnika Udemy Git course
Opis podsetnika lecture 8
Vreme (hh:mm) 17:15
Vreme notifikacije (hh:mm) 17:15
Podsetnik uspesno dodat.
Izaberite opciju (ili upisite 4 za izlaz): 1
Naziv podsetnika Cas nemackog
Opis podsetnika skype; pripremiti pitanja
Vreme (hh:mm) 18:15
Vreme notifikacije (hh:mm) 18:00
Podsetnik uspesno dodat.
Izaberite opciju (ili upisite 4 za izlaz): 1
Naziv podsetnika Diplomski
Opis podsetnika dodati literaturu i poslati rad mentoru
Vreme (hh:mm) 19:30
Vreme notifikacije (hh:mm) 19:30
Podsetnik uspesno dodat.
Izaberite opciju (ili upisite 4 za izlaz): 1
Naziv podsetnika system update
Opis podsetnika
Vreme (hh:mm) 21:00
Vreme notifikacije (hh:mm) 21:00
Podsetnik uspesno dodat.
Izaberite opciju (ili upisite 4 za izlaz): 3
13:30|14:00|HR intervju|online webex
17:15|17:15|Udemy Git course|lecture 8
18:00|18:15|Cas nemackog|skype; pripremiti pitanja
19:30|19:30|Diplomski|dodati literaturu i poslati rad mentoru
21:00|21:00|system update|

```

Slika 23. Prikaz ispravnog dodavanja podsetnika

U slučaju izbora opcije 2 – *obriši podsetnik* izvršava se funkcija *obriši*. Brisanje se vrši na osnovu unetog naziva podsetnika. Ukoliko u tekstualnom fajlu postoji više podsetnika sa istim nazivom, svi njihovi zapisi će biti uklonjeni. Takođe, provera unetog naziva se vrši pozivom iste funkcije *provera* kao na slici 21.

```

34 function obriši(){
35     read -rp "Naziv podsetnika koji treba obrisati " naziv
36     provera $naziv
37     awk -F "|" -v rec="$naziv" 'tolower($3) == tolower(rec) { next } { print }' podsetnik.txt > tmp.txt
38     mv tmp.txt podsetnik.txt
39     echo "Podsetnik uspesno obrisao"
40 }

```

Slika 24. Funkcija za brisanje podsetnika

Za pretragu i brisanje korišćena je awk komanda za obradu teksta.

- **-v rec="\$naziv"**: Opcija definiše varijablu awk *rec* i dodeljuje joj vrednost sadržanu u promenljivoj *naziv*. Ovo omogućava korišćenje vrednosti promenljive u awk skripti.
- **tolower(\$3) == tolower(rec) { next }**: Proverava se da li je vrednost trećeg polja (kolona) u liniji jednaka vrednosti promenljive *rec*. **tolower** omogućava *case-insensitive* pretragu prebacivanjem kompletnog teksta u mala slova. Ako je uslov ispunjen, prelazi se na sledeću liniju bez dalje obrade (naredba **next**).

- **{ print }:** Ako se uslov nije ispunio (nije pronađeno podudaranje), ispisuje se cela linija u *tmp.txt* privremeni fajl
- **mv tmp.txt podsetnik.txt** linija preimenuje datoteku *tmp.txt* u *podsetnik.txt*, čime se originalni fajl zamenjuje privremeno kreiranim fajlom u kome se nalaze zapisi podsetnika čiji naziv ne odgovara nazivu koji je korisnik uneo.

Važno je napomenuti da se kao argument mora uneti kompletan naziv podsetnika tako da u potpunosti odgovara zapisu u tekstualnom fajlu. Izborom opcije 3 korisniku je omogućeno da u svakom trenutku prilikom korišćenja skripte ima uvid u sadržaj fajla kako bi uneo tačan naziv podsetnika koji je potrebno izbrisati, kao i da proveri da li je zapis zaista izbrisan.

```
Izaberite opciju (ili upisite 4 za izlaz): 3
13:30|14:00|HR intervju|online webex
17:15|17:15|Udemy Git course|lecture 8
18:00|18:15|Cas nemackog|skype; pripremiti pitanja
19:30|19:30|Diplomski|dodati literaturu i poslati rad mentoru
21:00|21:00|system update|
Izaberite opciju (ili upisite 4 za izlaz): 2
Naziv podsetnika koji treba obrisati hr intervju
Podsetnik uspesno obrisani
Izaberite opciju (ili upisite 4 za izlaz): 3
17:15|17:15|Udemy Git course|lecture 8
18:00|18:15|Cas nemackog|skype; pripremiti pitanja
19:30|19:30|Diplomski|dodati literaturu i poslati rad mentoru
21:00|21:00|system update|
```

Slika 25. Prikaz brisanja podsetnika

## 5. SKRIPTA ZA SLANJE NOTIFIKACIJA NA LOKALNI I/ILI UDALJENI RAČUNAR

U ovom poglavlju biće izloženo korisničko uputstvo uz prikaz rada i objašnjenje koda skripte koja na osnovu zapisa podsetnika iz fajla kreira i šalje notifikacije na lokalni i/ili udaljeni računar.

### 5.1. Programski kod skripte

---

```
1 #!/bin/bash
2
3 remote_user="ubuntu" #korisnicko ime udaljenog korisnika
4 remote_host="192.168.1.36" #ip adresa udaljenog korisnika ili DNS
5 send_local=false
6 send_remote=false
7
8 local_notify() {
9     notify-send "$naziv" "$opis"
10 }
11 remote_notify() {
12     ssh "$remote_user@$remote_host" "notify-send \"$naziv\" \"$opis\""
13 }
14
15 while getopts "lr" opcija; do          #biranje opcija -r, -l, -rl/lr (podrazumevano)
16     case $opcija in
17         l) send_local=true ;;
18         r) send_remote=true ;;
19         *) echo "Neispravna opcija"
20             exit 1 ;;
21     esac
22 done
23
24 while true; do
25     while IFS= read -r red; do
26         vreme_notifikacije=$(echo "$red" | cut -d "|" -f1)
27         trenutno_vreme=$(date +%H:%M)
28
29         if [[ $trenutno_vreme = $vreme_notifikacije ]]; then
30
31             naziv=$(echo "$red" | cut -d "|" -f2,3 | sed 's/|/ /')
32             opis=$(echo "$red" | cut -d "|" -f4)
33
34             if $send_local; then
35                 local_notify
36             fi
37             if $send_remote; then
38                 remote_notify
39             fi
40             if [[ ! $send_remote && ! $send_local ]]; then
41                 remote_notify
42                 local_notify
43             fi
44
45             fi
46         done < podsetnik.txt
47
48         sleep 60 #ponavlja se nakon 1 minuta
49 done &
```

---

Slika 26. Programski kod skripte za slanje notifikacija na udaljeni i/ili lokalni računar *send\_opt.sh*

## 5.2. Korisničko uputstvo i analiza koda

Na početku je potrebno definisati vrednosti promenljivih `remote_user` – korisničko ime udaljenog korisnika kome se šalju notifikacije i `remote_host` – IP adresa udaljenog korisnika. Takođe je potrebno omogućiti komunikaciju ova dva računara pomoću SSH (poglavlje 3.2).

Pri pokretanju skripte moguće je izabrati opcije (slika 27):

- **-l** notifikacija se prikazuje samo na lokalnom računaru
- **-r** notifikacija se prikazuje samo na udaljenom računaru
- **-lr/rl (default)** notifikacija se prikazuje i na lokalnom i na udaljenom računaru
- U slučaju izbora nepostojećih opcija, skriptu je nemoguće pokrenuti

Na početku, vrednosti promenljivih `send_local`, `send_remote` postavljene su na `false`. U zavisnosti od izabrane opcije te vrednosti se menjaju.

```
15 while getopts "lr" opcija; do      #biranje opcija -r, -l, -rl/rl (podrazumevano)
16     case $opcija in
17         l) send_local=true ;;
18         r) send_remote=true ;;
19         *) echo "Neispravna opcija"
20             exit 1 ;;
21     esac
22 done
```

Slika 27. Opcije

Potrebno je da skripta radi neprestano, zbog čega se uvodi beskonačna `while true` petlja u kombinaciji sa `&`, što omogućava da se izvršava u pozadini (čak i pri zatvaranju terminala) pri čemu je komandna linija slobodna za dalje korišćenje. Podešeni su `sleep 60` intervali koji označavaju da se petlja ponavlja svakog minuta (ovaj interval se može modifikovati, tj. smanjiti/povećati u zavisnosti od potreba). Periodično izvršavanje skripte se na drugi način moglo realizovati definisanjem cron zadatka (slika 5). U tom slučaju se izostavlja `while` beskonačna petlja.

```
24 while true; do
25     while IFS= read -r red; do
26         vreme_notifikacije=$(echo "$red" | cut -d "|" -f1) #vreme notifikacije je u prvoj koloni fajla
27         trenutno_vreme=$(date +%H:%M)
28
29         if [[ $trenutno_vreme == $vreme_notifikacije ]]; then #proveravanje da li je vreme za slanje notifikacije
30
31             naziv=$(echo "$red" | cut -d "|" -f2,3 | sed 's/|/-/') #spajanje 2. i 3. kolone fajla → hh:mm-NAZIV
32             opis=$(echo "$red" | cut -d "|" -f4)
33
34             if $send_local; then
35                 local_notify
36             fi
37
38             if $send_remote; then
39                 remote_notify
40             fi
41
42             if [[ ! $send_remote && ! $send_local ]]; then
43                 remote_notify
44                 local_notify
45             fi
46
47             fi
48         done < podsetnik.txt
49
50         sleep 60 #ponavlja se nakon 1 minuta
51 done &
```

Slika 28. Glavni deo skripte – provera vremena i slanje notifikacija



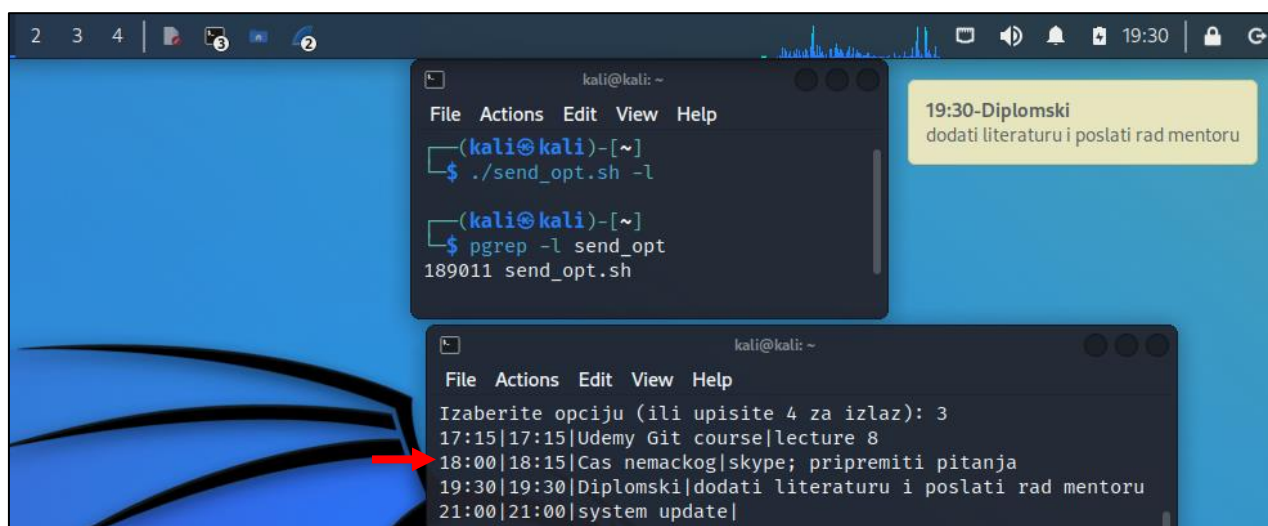
Najpre, **while IFS= read -r red; do** predstavlja početak petlje koja čita redove iz *podsetnik.txt* fajla i dodeljuje ih promenljivoj *red*. Zatim se iz reda izdvaja prva kolona koja predstavlja vreme u koje je potrebno poslati notifikaciju korišćenjem *cut* komande:

Korišćenjem **date +%H:%M** uzima se trenutno vreme u formatu hh:mm i dodeljuje se promenljivoj *trenutno\_vreme*. Zatim se vrednost trenutnog vremena upoređuje sa vremenom notifikacije. Ukoliko se ova dva vremena poklapaju, iz teksta se pomoću *cut* komande izdvajaju i ostala polja iz *reda*, kako bi se formirala desktop notifikacija. Naziv podsetnika koji se prikazuje na notifikaciji se formira dohvatanjem druge (vreme) i treće (naziv) kolone, a u kombinaciji sa *sed* komandom se delimiter “|” iz originalnog teksta zamenjuje sa „-“. Opis podsetnika ostaje nepromenjen. Na taj način se formira krajnji izgled notifikacije.

U slučaju da je skripta pokrenuta uz opciju *-l* notifikacije se prikazuju samo na lokalnom računaru. Izvršava se funkcija *local\_notify* – slika 29.

```
8 local_notify() {                               #slanje notifikacije samo na lokalni racunar
9     notify-send "$naziv" "$opis"
10 }
```

Slika 29. Funkcija *local\_notify*



Slika 30. Prikaz rada opcije *-l*, notifikacija samo na lokalnom računaru

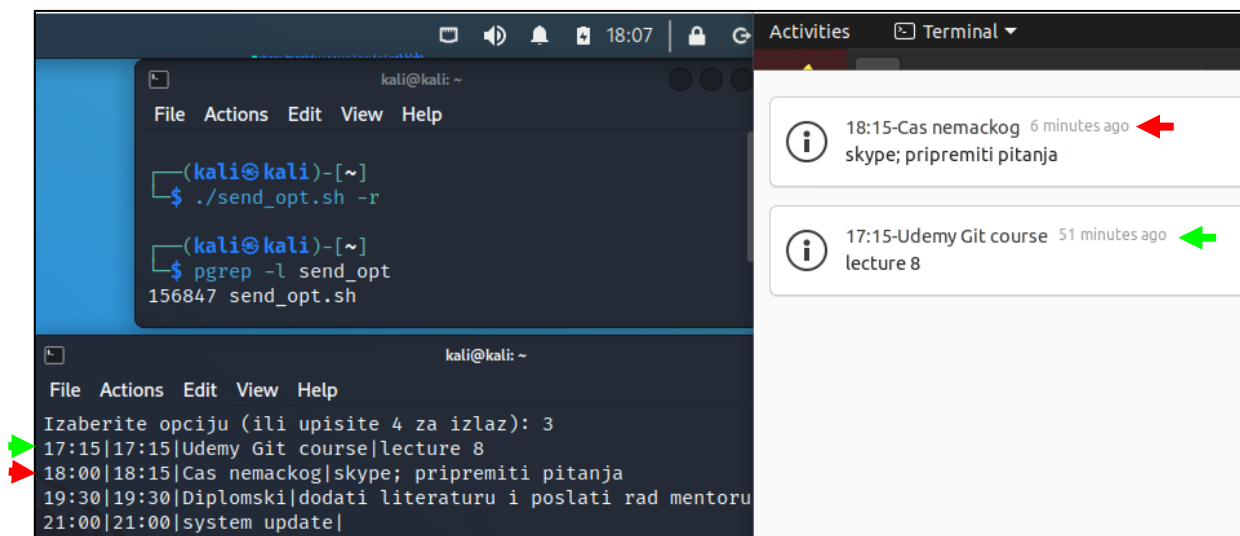
U slučaju da je skripta pokrenuta uz opciju *-r* notifikacije se prikazuju samo na udaljenom računaru. Izvršava se funkcija *remote\_notify*:

```
11 remote_notify() {                             #slanje notifikacije samo na udaljeni racunar
12     ssh "$remote_user@$remote_host" "notify-send \"$naziv\" \"$opis\""
13 }
```

Slika 31. Funkcija *remote\_notify*

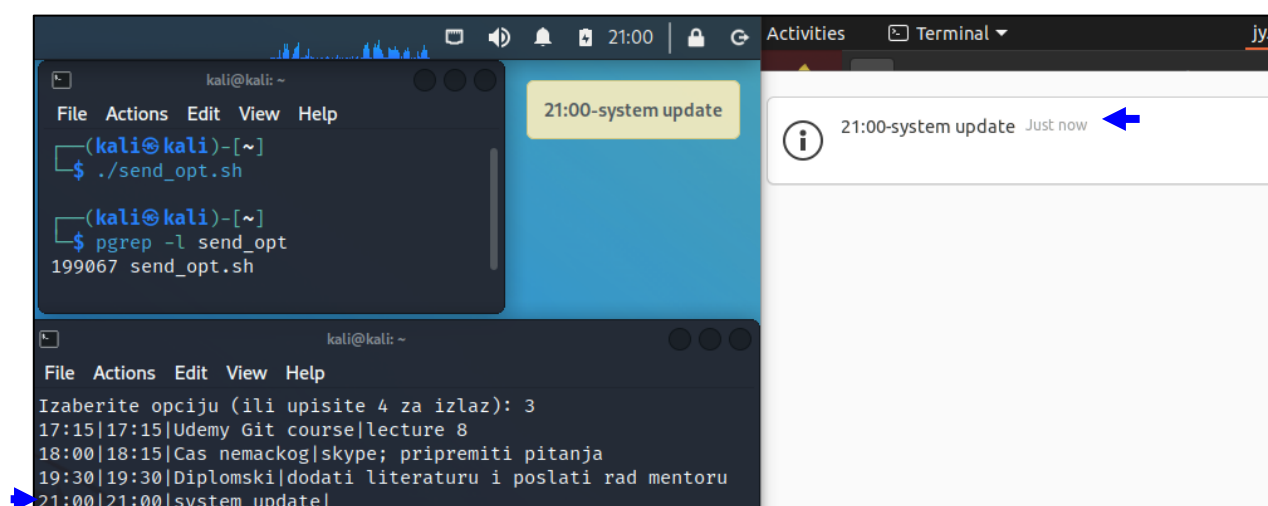
Izvršava se komanda *notify-send* na udaljenom računaru putem SSH sesije. Koristi se *escape* znak “\” kako bi se označili navodnici unutar komande *notify-send*, jer su već prisutni navodnici oko cele SSH komande. *\$naziv* i *\$opis* su unutar dvostrukih navodnika kako bi se očuvala celovitost naslova i poruke notifikacije, ako sadrže razmake ili posebne znakove.





Slika 32. Prikaz rada opcije -r, notifikacije na udaljenom računaru (ubuntu)

U slučaju da je skripta pokrenuta bez opcija ili sa *-rl*, *-lr* izvršavaju se obe navedene funkcije.



Slika 33. Prikaz default opcije (-rl/lr), notifikacija na oba računara

Kada skripta radi u pozadini sistema, nije moguće prekinuti je na uobičajen način koristeći CTRL+C. U takvim slučajevima, morate pronaći ID procesa (PID) i koristiti komandu "kill PID" kako biste zaustavili izvršavanje skripte. Komanda **pgrep -l send\_opt** se koristi za pronalaženje procesa koji se izvršavaju sa nazivom "send\_opt" i prikazuje rezultate sa listom identifikatora procesa (PID).

## 6. ZAKLJUČAK

Skripte su izuzetno korisne jer omogućavaju automatizaciju nizova komandi koje bi inače zahtevale ručno unošenje u komandni prompt. One omogućavaju realizaciju funkcionalnosti koje bi bilo teško ili nemoguće postići ručnim unosom komandi, čime nam pružaju veću efikasnost, pouzdanost i praktičnost. Jednostavnost sintakse Bash-a olakšava pisanje i održavanje skripti. Uz relativno malo linija koda može se postići njihova velika funkcionalnost. Kroz implementaciju Bash skripti za dodavanje, brisanje i slanje podsetnika, demonstrirana je moć ovog jezika i njegov doprinos u olakšavanju upravljanja vremenom i organizaciji obaveza.

Automatsko formatiranje podataka prilikom unosa, kao i pretraga podsetnika koje je potrebno izbrisati značajno doprinosi uštedi vremena, smanjenju grešaka prilikom unosa i ne zahteva veliki trud korisnika prilikom unosa i pretrage. Takođe, koncept podsetnika gubi svaki smisao kada se šalje „ručno“, jer bi zahtevao konstantno praćenje vremena od strane korisnika kome je i potreban podsetnik, dizajniranje notifikacija i druge detalje. Umesto toga skripta preuzima te zadatke na sebe i automatski izvršava sve korake za slanje podsetnika na osnovu unapred definisanih zapisa.

Ove skripte je moguće prilagoditi i unaprediti na razne načine. Na primer, uz brisanje i unos, može se implementirati i opcija koja bi omogućila ažuriranje vremena podsetnika, povezivanje podsetnika sa određenim zadacima ili događajima. Takođe, može omogućiti da se zapisi podsetnika automatski brišu nakon slanja notifikacije. Notifikacije bi se mogle prikazivati i na većem broju uređaja ili se mogu slati e-mailom, ili na neki drugi način. Uz to, notifikacije se mogu i drugačije formatirati korišćenjem odgovarajućih opcija koje nudi *notify-send* funkcija.

Fleksibilnost i automatizacija čine Bash izuzetno moćnim alatom za upravljanje podacima, sistemima i zadacima. Korisnicima je omogućeno da stvaraju svoje personalizovane alatke i rešenja, prilagođene njihovim specifičnim potrebama. Ova sposobnost pruža slobodu da se kreira funkcionalnost koja se tačno uklapa u zahteve korisnika, bez potrebe za zavisnošću od gotovih softvera i aplikacija. To ga čini važnim alatom za sistem administratore, programere i sve druge korisnike Linux sistema. Automatizacija rutinskih zadataka oslobađa korisnike od ponavljanja istih radnji, što značajno doprinosi na smanjenju grešaka, pomaže u očuvanju integriteta podataka i poboljšanju efikasnosti rada.

# LITERATURA

1. Z. Čiča, Mrežna administracija i programiranje, Akademska Misao, 2022.
2. <http://alas.matf.bg.ac.rs/~mr18307/bash.html>
3. [https://www.hostinger.com/tutorials/ssh-tutorial-how-does-ssh-work#How\\_Does\\_SSH\\_Work](https://www.hostinger.com/tutorials/ssh-tutorial-how-does-ssh-work#How_Does_SSH_Work)
4. <https://www.digitalocean.com/community/tutorials/understanding-the-ssh-encryption-and-connection-process>

## SPISAK SLIKA

|                                                                                                                       |    |
|-----------------------------------------------------------------------------------------------------------------------|----|
| Slika 1. shebang linija .....                                                                                         | 4  |
| Slika 2. Pozicioni argumenti .....                                                                                    | 5  |
| Slika 3. Interaktivni unos .....                                                                                      | 5  |
| Slika 4. Dodeljivanje izvršne dozvole.....                                                                            | 5  |
| Slika 5. Zapis cron zadatka .....                                                                                     | 6  |
| Slika 6. Periodično izvršavanje skripte pomoću beskonačne petlje.....                                                 | 6  |
| Slika 7. Sintaksa case, if, while.....                                                                                | 8  |
| Slika 8. Three-way handshake .....                                                                                    | 10 |
| Slika 9. Diffie-Hellman algoritam .....                                                                               | 11 |
| Slika 10. Autentifikacija klijenta .....                                                                              | 12 |
| Slika 11. Wireshark PCAP – uspostava SSH sesije (TCP port 22) .....                                                   | 13 |
| Slika 12. Instalacija SSH servera i klijenta .....                                                                    | 13 |
| Slika 13. Provera statusa SSH.....                                                                                    | 13 |
| Slika 14. Otvaranje SSH porta 22 .....                                                                                | 14 |
| Slika 15. Generisanje RSA ključeva .....                                                                              | 14 |
| Slika 16. Kopiranje ključa na SSH server.....                                                                         | 14 |
| Slika 17. Uspešna uspostava SSH veze .....                                                                            | 15 |
| Slika 18. Programski kod skripte za unos, brisanje i pregled podsetnika - <i>podsetnik.sh</i> .....                   | 17 |
| Slika 19. Pokretanje skripte, prikaz menija i primer pogrešnog izbora opcije .....                                    | 17 |
| Slika 20. Funkcija za dodavanje podsetnika .....                                                                      | 17 |
| Slika 21. Provera unosa naziva podsetnika .....                                                                       | 18 |
| Slika 22. Provera formata vremena.....                                                                                | 18 |
| Slika 23. Prikaz ispravnog dodavanja podsetnika.....                                                                  | 19 |
| Slika 24. Funkcija za brisanje podsetnika .....                                                                       | 19 |
| Slika 25. Prikaz brisanja podsetnika .....                                                                            | 20 |
| Slika 26. Programski kod skripte za slanje notifikacija na udaljeni i/ili lokalni računar <i>send_opt.sh</i><br>..... | 21 |
| Slika 27. Opcije.....                                                                                                 | 22 |
| Slika 28. Glavni deo skripte – provera vremena i slanje notifikacija .....                                            | 22 |
| Slika 29. Funkcija <i>local_notify</i> .....                                                                          | 23 |
| Slika 30. Prikaz rada opcije -l, notifikacija samo na lokalnom računaru.....                                          | 23 |
| Slika 31. Funkcija <i>remote_notify</i> .....                                                                         | 23 |
| Slika 32. Prikaz rada opcije -r, notifikacije na udaljenom računaru (ubuntu) .....                                    | 24 |
| Slika 33. Prikaz default opcije (-rl/lr), notifikacija na oba računara.....                                           | 24 |

## SPISAK SKRAĆENICA

|         |                                      |
|---------|--------------------------------------|
| Bash    | <i>Bourne Again Shell</i>            |
| CLI     | <i>Command Line Interface</i>        |
| GUI     | <i>Graphical User Interface</i>      |
| SSH     | <i>Secure Shell</i>                  |
| TCP     | <i>Transmission Control Protocol</i> |
| SYN     | <i>Synchronize</i>                   |
| SYN-ACK | <i>Synchronize-Acknowledgement</i>   |
| ACK     | <i>Acknowledgement</i>               |
| MD5     | <i>Message Digest 5</i>              |
| MAC     | <i>Message Authentication Code</i>   |
| SHA     | <i>Secure Hash Algorithm</i>         |
| IFS     | <i>Internal Field Separator</i>      |
| PCAP    | <i>Packet Capture</i>                |